Date of create: - 01/02/2026 Creator name: Shaik Abdul Rehman

Fake Social Media Profile Detection Using Machine Learning

Project Report & Technical Documentation | Data Science & Machine Learning

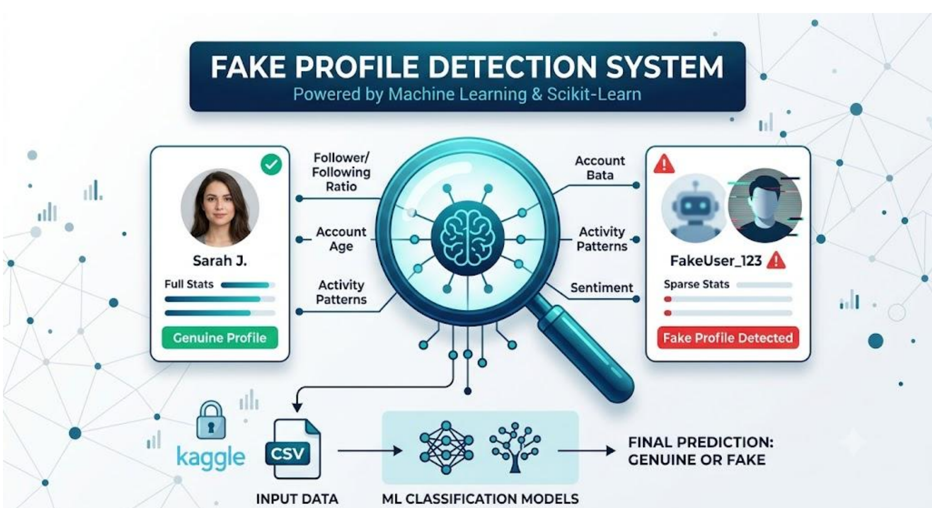
Prepared By: Project Contributor	**Platform:** Google Colab / Python 3
Dataset: debanshupal/fake-profile-detection-dataset	**Repository:** fake-profile-detection-ml

Abstract

The rapid expansion of online social networks has brought unprecedented connectivity, but it has also created major vulnerabilities regarding automated spam, identity spoofing, and malicious bot activity. Fake profiles distort user engagement metrics, spread disinformation, and pose phishing risks to genuine users. This project implements an end-to-end Machine Learning pipeline in Python to identify and classify fake social media accounts based on behavioral metadata, account age, activity intensity, and follower dynamics. By utilizing structured feature extraction and supervised classification algorithms, the model provides an automated mechanism to detect suspicious accounts prior to platform abuse.

1. System Architecture & Workflow

The project architecture spans dataset ingestion via the Kaggle API, automated feature engineering, model training, and evaluation. The core logic relies on identifying discrepancies between genuine account usage patterns and automated bot behaviors.



2. Dataset Overview & Key Attributes

The model was trained on the Kaggle/fake-profile-detection-dataset. This dataset contains annotated account profiles split into genuine users and fake/bot accounts. Rather than analyzing raw text or heavy multimedia, the classification relies primarily on structured account metadata, making it computationally lightweight and scalable.

Key Feature Categories

- **Follower to Following Ratio:** Automated accounts typically exhibit extreme ratios—either following thousands of accounts with zero followers or maintaining artificial follow-for-follow networks.
- **Account Age & Tenure:** Newly created accounts with immediate high activity bursts strongly indicate automated spam bots.
- **Profile Completeness:** Genuine accounts tend to have custom avatars, written biographies, linked locations, and verified contact details. Fake profiles frequently lack complete bios.
- **Activity & Engagement Patterns:** Analysis of posting frequency, comment density, and baseline interaction parameters across user histories.

3. Methodology & Implementation Pipeline

Step 1: Secure Data Ingestion

To follow security best practices, access tokens and private credentials (such as `kaggle.json`) are injected via environment variables or runtime secrets rather than hardcoding credentials into source notebooks. The dataset is fetched programmatically at runtime using the Kaggle API.

Step 2: Preprocessing & Feature Scaling

Raw data fields were inspected for missing values, duplicate user IDs, and structural anomalies. Continuous numerical attributes (such as follower counts and account activity spans) were standardized using `StandardScaler` to balance feature contributions. Categorical variables were binary-encoded.

Step 3: Supervised Model Training

Multiple supervised classification models were evaluated using Scikit-Learn, including Random Forest Classifiers and Logistic Regression. The dataset was split into 80% training and 20% testing subsets to cross-validate model generalizability.

4. Execution Summary

Pipeline Stage	Implementation Details
Data Source	Kaggle Dataset API (debanshupal/fake-profile-detection-dataset)
Preprocessing	Imputation, Feature Scaling (StandardScaler), Encoding
Core Models	Random Forest Classifier, Logistic Regression

5. Reproduction & Setup Instructions

Step 1: Clone the repository to your local machine or open directly in Google Colab.

Step 2: Ensure Python dependencies (`pandas`, `scikit-learn`, `kaggle`, `matplotlib`) are installed via `pip install -r requirements.txt`.

Step 3: Configure Kaggle API authentication key via environment variable or Colab Secrets (`KAGGLE_KEY`).

Step 4: Execute the notebook sequentially to run data loading, feature transformation, training, and metrics evaluation.